

One-time trial codes

The prospect trial is now gated by single-use codes — `QFWORK-TRIAL-a308b5` — instead of one shared code. Each code is good for one live call, on any device, and is refused afterwards.

The problem this fixes

`TRIAL_RUNS_SALES=1` was enforced on the **browser session**, not the person. `/api/access` minted a fresh session on every code entry, and the run counter lived on that session. So a prospect who opened a private tab and re-typed the same shared code got a brand new session with a full allowance — and another set of conversational-video minutes on our bill.

The cap held against an accidental reload. It did not hold against anyone who tried.

The fix is one structural move: the run counter now lives on the code, not the session. A reload, a private tab and a second device all mint different sessions, but they all present the same code, and the code is what gets charged.

What changed

1. One-time codes

`ACCESS_CODES_TRIAL` holds a comma-separated list of issued codes. Each carries its own run counter, capped by `TRIAL_RUNS_SALES` as before. They resolve to the existing **sales tier**, so report redaction, the run cap and the consultation card are untouched — only the accounting moved.

Matching ignores case, whitespace **and dashes**, because these get typed off an email and usually on a phone. `QFWORK-TRIAL-A308B5`, `qfwork trial a308b5` and `QFWORKTRIALA308B5` are the same code.

2. A spent code is refused at the gate

Previously a used-up trial only failed at the moment the call was started. Now `/api/access` returns `403 trial-spent` as soon as the code is entered, carrying `CALENDLY_URL` with it, and the gate renders a booking link under the message rather than inviting another attempt.

The IP attempt limiter is also now cleared for any **recognised** code, spent or not. Recognising a code proves it is not a guessing loop; without this, a prospect re-entering their own used code a few times would trip the limiter and get "too many attempts" instead of the message that explains what actually happened.

3. Spends survive a restart

What each code has spent is written to `trial-codes-used.json` (override with `TRIAL_CODES_STATE`), written tmp-then-rename so a crash mid-write cannot truncate it. A read-only filesystem warns once at boot and carries on in memory rather than failing.

The issued list stays in the environment. Only the tally is state, so there is still no database.

4. Link-through codes

A recipient can be sent straight in:

```
https://your-host/exam.html?code=QFWORK-TRIAL-b4ed55
```

The gate fills itself in, submits, and strips the code from the address bar — these links get screen-shared, and a stale code in browser history is just confusing.

5. `ACCESS_CODE_SALES` is deprecated

The old shared prospect code still works, so nobody holding one is locked out mid-trial. It keeps its per-session cap and therefore its reload weakness. Blank it once nobody is still holding it.

How to use it

1. Generate a batch.

```
node -e "for(let i=0;i<25;i++)console.log('QFWORK-TRIAL-  
' + require('crypto').randomBytes(3).toString('hex'))"
```

2. Register them. Paste the codes comma-separated into `ACCESS_CODES_TRIAL` — in `.env` locally, and in the Railway Variables tab for production. **This step is what makes them real.** A generated code that was never registered is rejected like any other invalid string.

```
ACCESS_CODES_TRIAL=QFWORK-TRIAL-b4ed55,QFWORK-TRIAL-4a47c6,QFWORK-TRIAL-  
e64250
```

Confirm at boot:

```
Trial : 20 one-time code(s) · 0 used · 20 left (1 run(s) each)
```

3. Send one code to one person — as a bare code, or as a `?code=` link.

Rules of thumb

Do	Why
Send each code to exactly one person	The first to use it burns it for everyone
Keep your own record of code → recipient	The server tracks <i>that</i> a code was spent, never <i>who</i> spent it

Do	Why
Append to the list to add more	Existing spends are keyed by code, so nothing already used resets
Remove a code to revoke it	It stops working immediately, spent or not
Don't re-run the generator to "get a code"	It produces a fresh unregistered set with no relation to what is live

Two decisions worth knowing about

A code is charged when the call starts, not when it is entered. Entering a code opens a session; the burn happens inside `/api/conversation`, the moment conversational-video billing begins. So someone who types their code and closes the tab has not lost their try, and a failed call start still refunds. Re-entering a claimed-but-unspent code simply resumes.

The tier system did the work. A one-time code resolves to the existing `sales` tier, so `redactForTier()`, the locked report cards, the run cap and the booking card are all unchanged. The whole feature is roughly 70 lines in `access.js` and 10 in `server.js` precisely because it changed *where a number is stored* rather than adding a parallel mechanism.

Impact

Area	Before	After
Second call per prospect	Open a private tab, re-enter the shared code	Refused — same code, already spent
Codes issued	One, shared by everyone	One per prospect, individually revocable
Spent state after restart	Lost with the session	Persisted to disk
Spent code experience	Fails at the call, generic message	Refused at the gate, with a booking link
Getting in on a phone	Type the code by thumb	A link that fills the gate in
Revoking access	Change the code, locking out everyone	Remove one code

No change to the conversation, the grading rubrics, the voice analysis, the report, or the demo tier.

Files

Modified

File	Change
<code>access.js</code>	<code>normCode()</code> ; the issued-code registry; the persisted spend tally; <code>checkCode()</code> now returns <code>{ tier, code }</code> ; sessions carry their code; <code>consumeRun()</code> / <code>runsLeft()</code> charge the code; <code>trialSpent()</code> , <code>trialStats()</code>
<code>server.js</code>	<code>403 trial-spent</code> branch in <code>/api/access</code> with the booking link; attempt limiter cleared for recognised codes; trial line in the boot banner
<code>public/access-gate.js</code>	Spent-code refusal with booking link; <code>?code=</code> prefill and address-bar cleanup; placeholder shows the real code shape
<code>.env</code> / <code>.env.example</code>	<code>ACCESS_CODES_TRIAL</code> , <code>TRIAL_CODES_STATE</code> ; <code>ACCESS_CODE_SALES</code> marked deprecated
<code>.gitignore</code>	<code>trial-codes-used.json</code> — runtime state, not configuration
<code>README.md</code>	Access section rewritten for the three code kinds and the persistence model

Generated at runtime

File	Purpose
<code>trial-codes-used.json</code>	Which codes are spent. Git-ignored. Safe to delete — it resets every code to unused

Before you deploy

1. **Add `ACCESS_CODES_TRIAL` to Railway.** The 20 codes are in local `.env` only; `.env` is git-ignored and excluded by `.railwayignore`, so it never reaches production.
2. **Decide about `ACCESS_CODE_SALES`.** It is still set to `QFWORK-TRIAL-2026` and still has the reload weakness. Blank it when nobody is holding it. Note that it *looks* like a one-time code but is not one.
3. **Set `CALENDLY_URL`** if it is still blank — the spent-code refusal has nowhere to send people without it.
4. **Consider a Railway volume.** Without one, `trial-codes-used.json` lives in the container and is rebuilt on redeploy, so every code returns to unused. Attach a volume at `/data` and set `TRIAL_CODES_STATE=/data/trial-codes-used.json`. Config only — no code change.

Known limits

- **Spends reset on redeploy without a volume.** Resetting requires *you* to deploy; a prospect cannot trigger it and cannot see when it happens. That is a real gap, but a much smaller one than a private tab.
- **No per-code expiry, label or issue date.** A code is live until you remove it from the list. All three are additive later.
- **No record of who holds which code.** Deliberate — it is a demo gate, not a login. Track the mapping yourself.

- **A leaked code is worth exactly one call**, which is the main improvement over a leaked shared code.
-

Verified

Code recognised with messy case, spaces and missing dashes; unknown and empty codes rejected; a spent code refused at the gate with **403 trial-spent** and the booking URL; a second session on a spent code refused, which is the private-tab case; an unrelated code unaffected; the spend surviving into a fresh process reading the same state file; the boot banner counting 20 codes and reporting used and remaining correctly.

Regressions checked: the demo code still uncapped and reusable, and the legacy shared sales code still capped per session and still reusable.

The live Tavus call path was not exercised end to end, since doing so bills real conversational-video minutes. The burn logic is covered by the module tests; the first real trial call is where to confirm it in place — the **used** count in the boot banner is the tell.